

8-bit Custom RISC CPU

Technical Design CPU Architecture

Project Name: 8-bit CPU
Document Version: Rev A
Date: September 19, 2026

Author: Vrontos Dimitrios
End Goal: FPGA Implementation
Architecture: Load-Store/Harvard model

1. Executive Summary

This document outlines the end-to-end design, verification, and physical implementation of a custom 8-bit RISC CPU, to be implemented on an FPGA hardware platform. Built upon a classic Load-Store architecture and utilizing a Harvard memory model with physically separate instruction (ROM) and data (RAM) spaces, the processor is engineered through five distinct execution phases:

- **Datapath & Core Execution Subsystems:** Implementation of an 8-bit Arithmetic Logic Unit (ALU), a 4-register general-purpose Register File (R0–R3) for fast state storage, and a Program Counter (PC) with sequential and direct jump capabilities.
- **Control Unit & Memory Integration:** Interfacing ROM/RAM blocks with a centralized Control Unit designed to decode 16-bit instruction words, drive critical control flags (e.g., ALU_Control, Write_Enable), and support conditional branching using ALU Zero (Z) and Carry (C) status flags.
- **Memory-Mapped I/O & Peripherals:** Development of a LED Matrix and button-based controller driven by video memory (VRAM).
- **Instruction Set Architecture (ISA) & Game Software:** Specification of custom assembly instruction opcodes and prevention of data collision logic.
- **FPGA Synthesis & Hardware Deployment:** Pin-mapping physical I/O resources (VGA signals, clock, switches) via Logisim Evolution, followed by HDL synthesis (VHDL/Verilog) and bitstream flashing onto the target FPGA development board.

2. First Design - CPU Foundational Structure

In this initial implementation, focus is placed on establishing the primary core architecture, specifically the foundational execution subcircuits detailed above and their interconnectivity. Secondary functional blocks, such as flag management, VGA interfacing, and peripherals, will be integrated in subsequent design iterations.

2.1 - Datapath

A datapath is a collection of functional units that perform data processing operations, register and data buses.

2.1.1 - Arithmetic Logic Units

The ALU is the main computational element of the CPU. For this specific implementation, it performs 8 separate computations, and chooses one through a multiplexer.

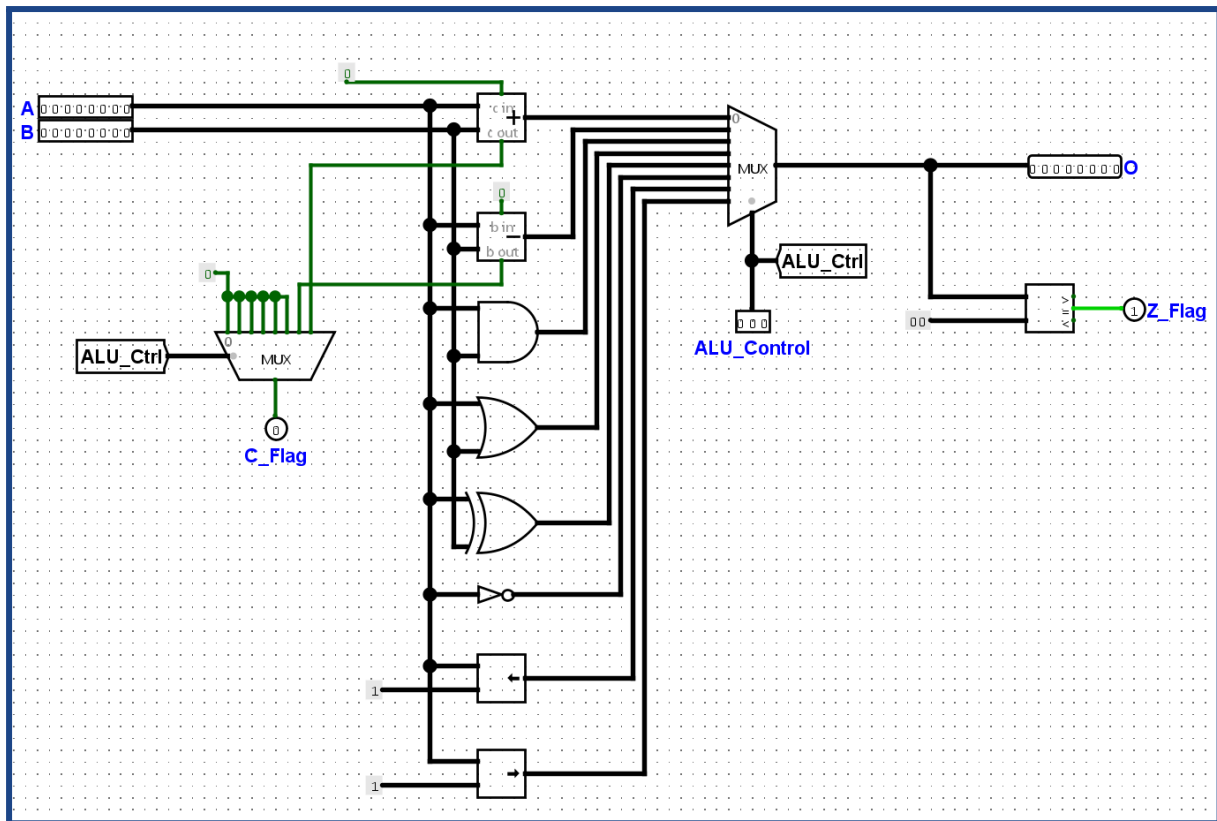
→ **Inputs and Outputs:**

- ◆ Data Entries: Two 8-bit numbers A and B
- ◆ Control unit: 3-bit control for the MUX (ALU_Control)
- ◆ Output: The output of the selected computation (O)
- ◆ Flags: Two 1-bit status flags, Z_Flag for zero output, and C_Flag for carry

→ **Operation Encoding & Implementation:**

ALU_Ctrl	Operation	Implementation / Description
000	Addition (ADD)	Uses an adder with inputs A and B, with carry input (c_in) tied to 0.
001	Subtraction (SUB)	Uses a subtractor with inputs A and B, with borrow input (b_in) tied to 0.
010	Logical AND	Implemented with an 8-bit AND gate for masking operations.
011	Logical OR	Implemented with an 8-bit OR gate for combining bits.
100	Logical XOR	Implemented with an 8-bit XOR gate, useful for equality comparisons or bit inversion.
101	Logical NOT	Uses a NOT gate taking only bus A as input, inverting all its bits.
110	Shift Left	Shifts the bits of A to the left by 1 position (with a constant 1 at the lower pin of the shifter).
111	Shift Right	Shifts the bits of A to the right by 1 position.

Table 1 - Operation Encoding



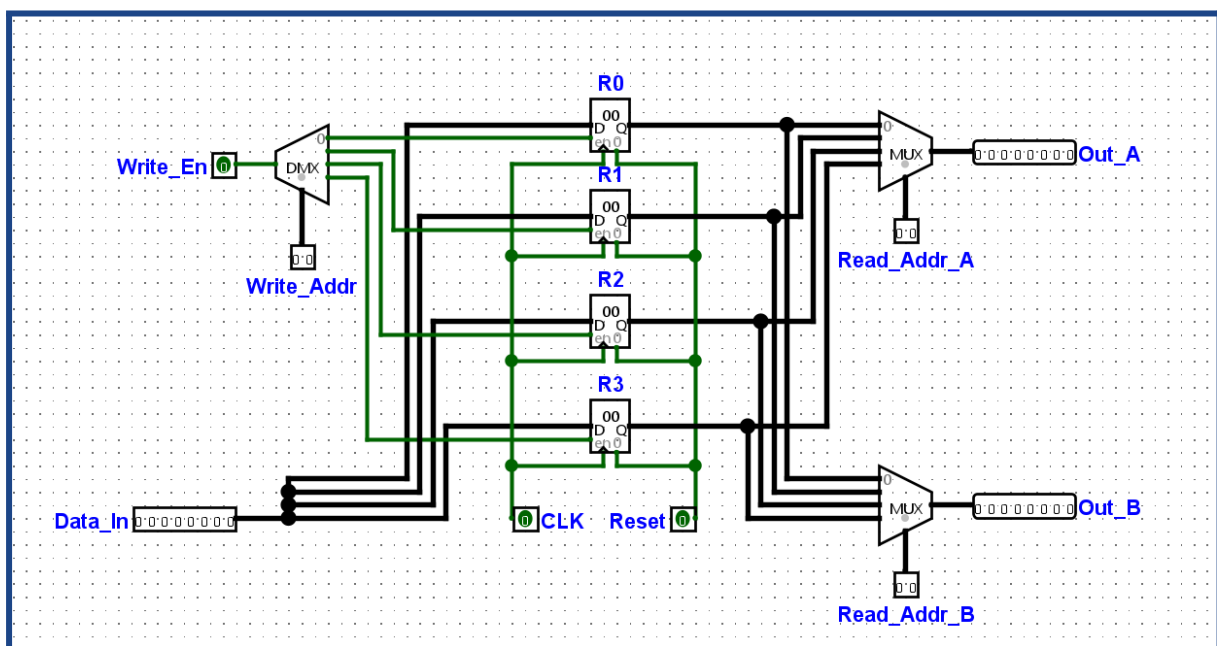
Subcircuit 1 - ALU

2.1.2 - Register File

The register file is a temporary and quick resource to memorize and manipulate data. In the chosen Load-Store architecture, the computational data aren't loaded onto the ALU directly through the RAM, but rather pass through the registers, computed, and then stored back to the registers.

- **Inputs and Outputs:** The Register File communicates with the other subcircuits (Control Unit, ALU, RAM) via the following control signals and 8-bit buses
- ◆ Read Buses (Read_Addr_A, Read_Addr_B): Receive the addresses of the two registers that must be read simultaneously.
 - ◆ Write Bus (Write_Addr): Receives the address of the register where the new result will be stored.
 - ◆ Data Input (Data_In): The 8-bit bus carrying the new data to be written. This data originates either from the ALU output (following an operation) or from the RAM (during a LOAD instruction), selected via the central MUX.
 - ◆ Data Outputs (Out_A, Out_B): Two independent 8-bit buses that transfer the contents of the selected registers directly to inputs A and B of the ALU.
 - ◆ Write Enable Signal (Write_En): When set to 1 (active), it permits the Data_In value to be stored in the selected register. When set to 0, the memory is read-only and protected from accidental modifications.

- ◆ Clock (CLK) and Reset: CLK synchronizes the writes to occur precisely on every system cycle, while Reset simultaneously clears the contents of all registers for system initialization.
- **Reading Procedure (Asynchronous):** When the system feeds the address to Read_Addr_A, the corresponding MUX pushes the correct Out_A.
- **Writing Procedure (Synchronous):** The input Data_In is loaded simultaneously on all four registers. Through the Write_Addr signal, the DMX sends the enable signal Write_En to the correct register. When the clock drops, only the enabled register has the new value loaded.
- ◆ The system of multiple registers helps the ALU have access to multiple useful data on demand, without searching the entire RAM.



Subcircuit 2 - Register File

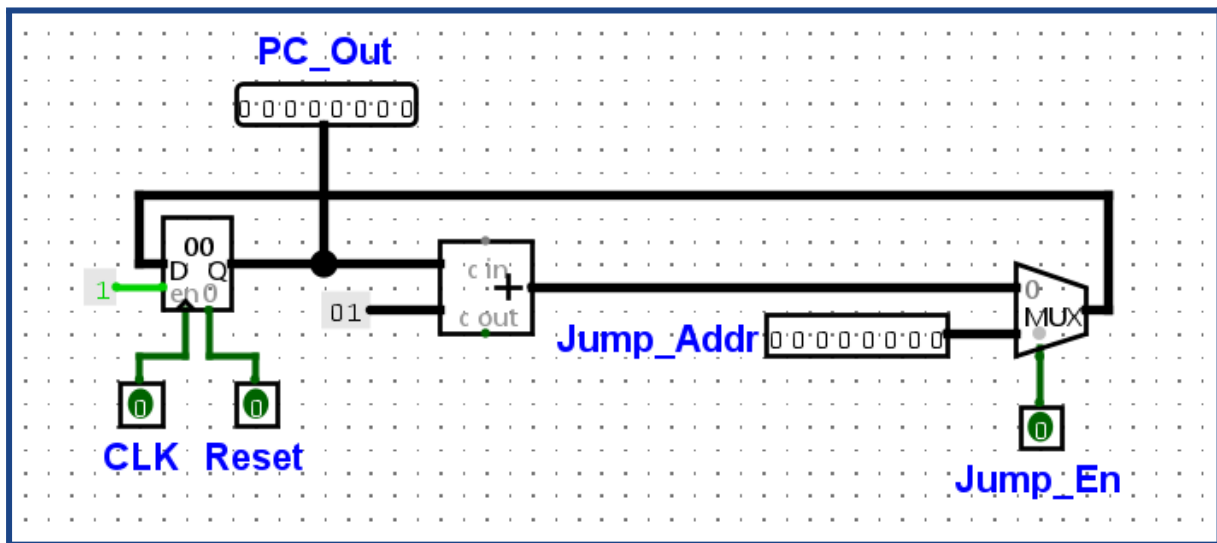
2.1.3 - Program Counter

The Program Counter is the unit that holds the memory address from the ROM that is next in line for computation. It has two functions. The first is to linearly go through the ROM's data one by one, upping its value at each iteration. The second is to jump at a needed address, breaking the sequential flow of the PC.

→ Inputs and Outputs:

- ◆ Clock (CLK): The primary timing signal. On every rising edge of the clock, the PC updates its value, allowing the system to proceed to the next step.
- ◆ Reset: A universal 1-bit input that, when activated, instantly clears the PC value (address 00). This ensures that upon system startup, the program will begin execution from the first line of the ROM.

- ◆ **Jump Enable (Jump_En):** The critical 1-bit control signal originating from the Control Unit. If it is 0, the PC simply increments its current value. If it is 1, the CPU enters jump mode.
 - ◆ **Jump Address (Jump_Addr):** An input bus that carries the new destination address directly from the Immediate field of the current instruction (via the Bit Extender).
 - ◆ **Address Output (PC_Out):** The output bus permanently connected to the Address pin of the ROM, dictating which instruction will be read.
- **Implementation:** The desired address is pulled by the ROM through a main register. Then, the content of the register is continuously incremented by one. The MUX is used to determine when this sequence will be stopped to make a jump. When $\text{Jump_En} = 1$, the Mux feeds the new address (Jump_Addr) to the register.



Subcircuit 3 - PC

2.2 - Memory Management (ROM/RAM)

The architecture of this CPU follows the Harvard model, with the main principle being the separation between the program's memory and the data's memory. This is being done for time efficiency, as it allows the CPU to read both variables and commands.

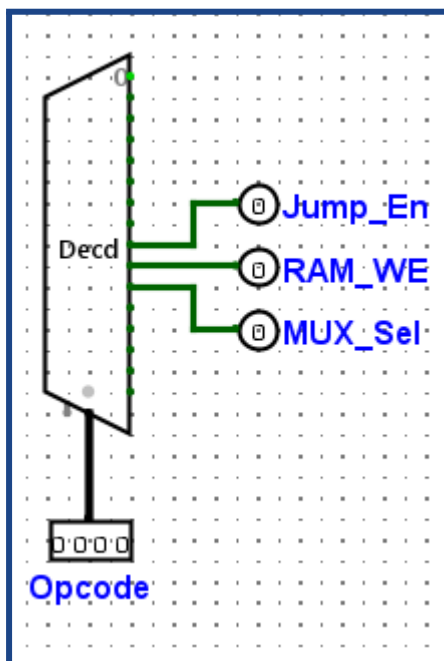
- **Read-Only-Memory:** It handles the storage related to the machine code. It remains constant throughout the duration of the program. Each line consists of the Opcode, which encodes the action needed, the register's addresses and possible constants. It works closely with the PC.
- **Random-Access-Memory:** It handles data that change as the program goes on. For example, it handles dynamic variables and operational states during execution. Another notable mention is one about the Data_In input for Register file. The Register file stores memory from either ALU's output, or from RAM. The choice is made by a multiplexer.

2.3 - Control Unit

The control unit is the coordinator of the CPU. It reads the Opcode from the ROM, and sends the necessary signals so that all the other units can take actions accordingly. The process is done through arbitrary encoding and a decoder. In this implementation, the control unit is responsible for three signals.

- Jump_En: Controls the PC. When activated, it allows for a jump to be executed
- RAM_WE: When activated, it allows for data to be written in the RAM.
- MUX_Sel: It controls the small multiplexer that gives the Register file its input.

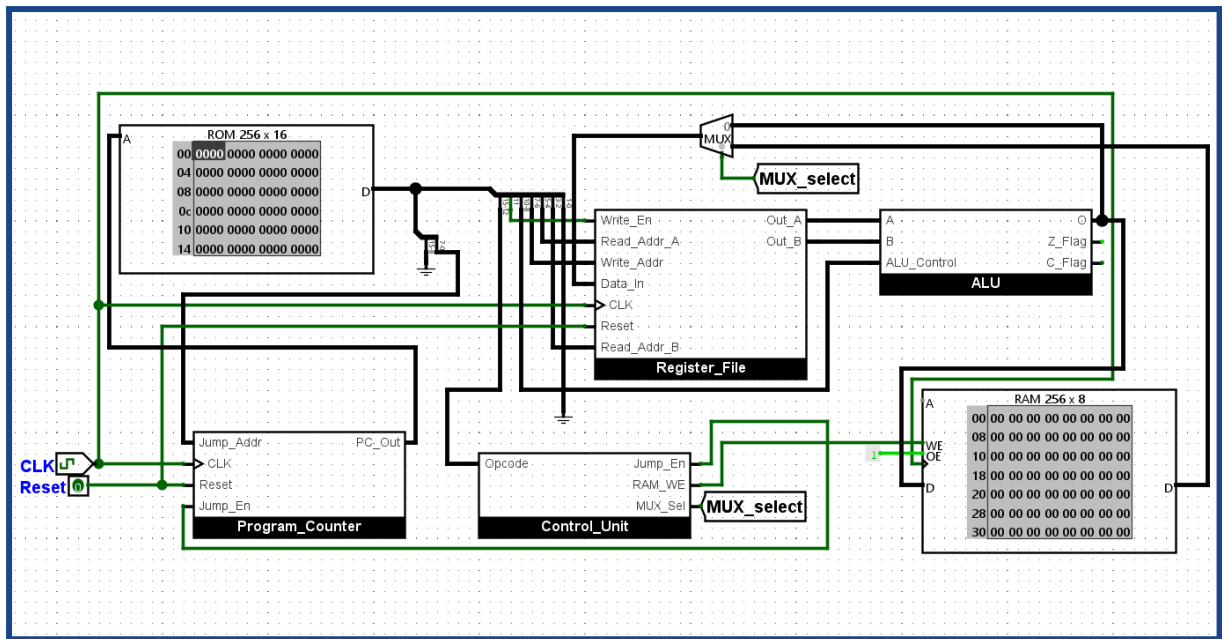
In the depiction of the whole circuit that will be displayed later on, you will see that the listed signals (Write_En, Read_Addr_A, Read_Addr_B, ALU_Control) have the encodings directly wired in the CPU circuit (Direct Hardware Mapping). Those signals could have been incorporated in the Control Unit. For the sake of simplicity and the straight-forwardness intended in this project, the current approach is preferred.



Subcircuit 4 - Control Unit

2.4 - Full Circuit and Tables

By assembling all the subcircuits, and according to the hardware mapping and control unit encodings, the full CPU prototype and the rules to navigate it are complete and shown below.



Circuit 1 - CPU prototype 1

Bit Range	Size	Field / Signal	Description / Notes
Bits [15–12]	4 bits	Opcode	Opcode for Control Unit decoding.
Bit [11]	1 bit	Write_En	Write Enable signal for Register File.
Bits [10–8]	3 bits	ALU_Control	Control signal directing ALU operation.
Bits [7–6]	2 bits	Write_Addr	Destination register address in Register File.
Bits [5–4]	2 bits	Read_Addr_A	Source register address A in Register File.
Bits [3–2]	2 bits	Read_Addr_B	Source register address B in Register File.
*Bits [7–0]	8 bits	Jump_Addr	In jump instructions (JMP), this lower 8-bit byte goes to the Bit Extender and then to the Program Counter as the target address (Jump_Addr). During a JMP instruction, register and ALU control bits are ignored.

Table 2 - ROM's 16-bit Layer Distribution

Opcode (Binary)	Hex	Instruction Name	Active Signals (Control Unit)	Function Description
0000	0	ALU_OP	None (000)	Execution of operation in the ALU (the result goes to Register File).
1000	8	JMP	Jump_En = 1	Jump to the new program address.
1001	9	STORE	RAM_WE = 1	Store data from Register File to RAM memory.
1010	A	LOAD	MUX_Sel = 1	Read data from RAM and load into the Register File.

Table 3 - Control Functions Table (Opcode - 4 bits)

Code (Binary)	Selected Register
00	R0
01	R1
10	R2
11	R3

Table 4 - Register Selection Table (2 bits per bus)

Code (Binary)	State	Result / Description
0	Disable	No change in registers (useful for comparisons, JMP, or STORE instructions).
1	Enable	Data_In is stored into the register indicated by Write_Addr.

Table 5 - Write Enable (1 bit)

ROM layer management - Data Collision Prevention

The CPU architecture is based on the philosophy of "Resource Sharing," which allows for the optimization of a limited 16-bit instruction bus. A fundamental architectural challenge is that the lower 8 bits of the instruction (Bits 0-7) are routed simultaneously to two different subsystems: they end up both in the Program Counter as a jump address (Jump_Addr) and in the Register File as register addresses (Read_Addr_A, Read_Addr_B, Write_Addr).

The resolution of this overlap is achieved through the strict coordination of Control Signals, which act as digital "safety gates." While the electrical data signals are transmitted in parallel to all subcircuits, the state change of a component is only permitted when its corresponding Enable signal is active:

- **Execution of Arithmetic/Logic Operations (ALU Operations):** When the Control Unit decodes an operation Opcode (e.g., 0000), it maintains the Jump_En signal at state 0. This "locks" the data input of the Program Counter, forcing it to ignore the lower 8 bits and proceed to the next sequential instruction. Simultaneously, the instruction sets Bit 11 (Write_En) to state 1. The Register File, having its write gate open, processes the 8 bits as valid register addresses, feeds the ALU, and safely stores the final result.
- **Execution of Jump Instructions (JMP Operations):** When the jump Opcode (1000) is decoded, the Control Unit sets the Jump_En signal to state 1. The Program Counter now accepts the lower 8 bits as the new execution address (Jump_Addr). Concurrently, the jump instruction format (Format 2) mandates that Bit 11 (Write_En) strictly holds the value of 0. Consequently, even though the Register File and the ALU continue to receive the same data and produce random (invalid) results in the background, the Register File remains locked. The denial of write access prevents the storage of invalid calculations, ensuring the integrity of the existing data.

This concludes the architectural specifications for the initial CPU prototype. As program execution logic is developed and refined, further iterative enhancements will be integrated into the design prior to its final design and physical deployment on the target FPGA board.

3. CPU Core Finalization

Following initial validation tests (such as basic loop counters) on the prototype, focus shifts to finalizing the CPU core to achieve a fully featured, universal algorithm execution engine. Transitioning from the simple initial implementation to a robust architecture capable of executing general-purpose software requires several key enhancements:

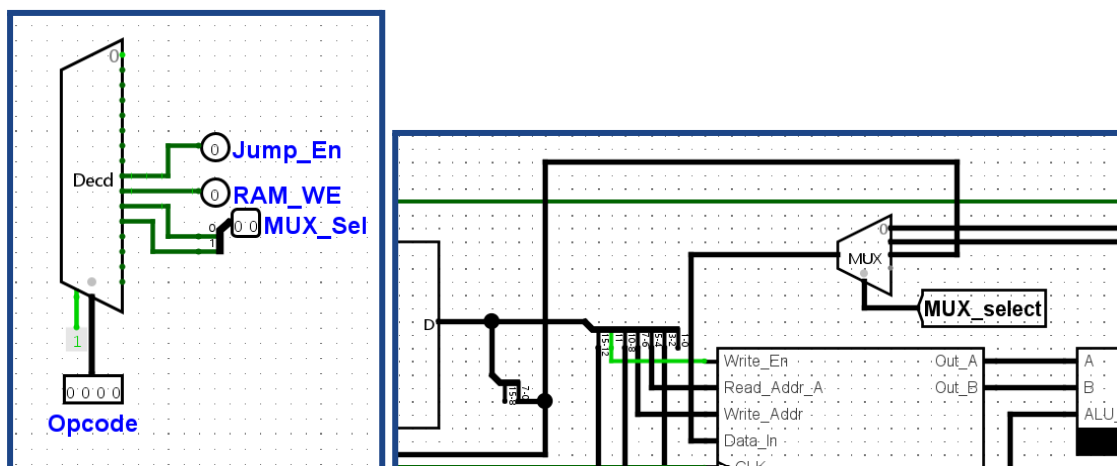
- Load Immediate System
- Flag Management and Conditional Jumps
- full RAM integration

3.1 - Load Immediate System (LDI)

The Load Immediate (LDI) instruction mechanism enables the direct loading of constant values (immediates) embedded within the ROM instruction word into a specified destination register in the Register File. This bypasses the memory read cycle from RAM, providing a fast and efficient way to initialize registers with constants, loop bounds, or initial memory addresses.

- **Instruction Encoding & Data Extraction:** The 16-bit instruction word allocates a dedicated Opcode field for LDI, a 2-bit field to define the destination register (Write_Addr), and an 8-bit immediate value field (Bits [7–0]). During execution, the control unit decodes the LDI opcode and routes the lower 8-bit byte directly from the ROM instruction payload toward the Register File data input bus.
- **Datapath Routing & Control Signals:** To route the immediate value into the selected register, the Control Unit activates the MUX control signal to select the immediate data path over the ALU output and RAM data buses. Simultaneously, the Write Enable signal (Write_En) is driven high to allow the 8-bit constant to be synchronously stored into the targeted register on the next rising clock edge.

The above planning is translated in the circuit by the following changes, as showcased below:



Testing: loop counters with initial values from ROM encoding - passed

- **ISA Update - LDI command:**

The syntax of the instruction is divided into the following fields:

- **Opcode (4-bit):** Reserved value B (1011).
- **Write_En (1-bit):** Register write enable.
- **Indifferents (3-bit):** Unused bits ignored during execution.
- **Data (8-bit):** The constant value to be loaded (Immediate Addressing).

The system architecture permanently routes bits 7 and 6 of the instruction to the write address (Write_Addr). Therefore, the destination register is not freely selected, but is strictly determined by the magnitude of the number being loaded.

For CPU programming, follow the mapping table below:

Value Range	Hexadecimal	Binary (Bits 7–6)	Destination Register
0 – 63	0x00 – 0x3F	00	R0
64 – 127	0x40 – 0x7F	01	R1
128 – 191	0x80 – 0xBF	10	R2
192 – 255	0xC0 – 0xFF	11	R3

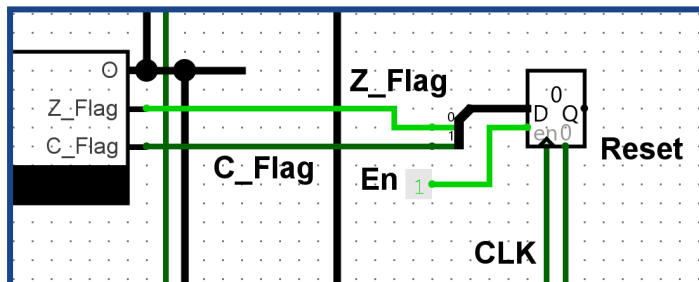
Table 6 - Register to value Correspondance

3.2 - Flag Management and Conditional Jumps

Flag management enables the CPU to perform conditional branching and dynamic decision-making based on runtime execution results. This subsystem integrates dedicated status flag registers (e.g., Zero and Carry flags generated by the ALU), modifies the Control Unit decoding logic to evaluate condition codes, and establishes conditional jump control paths for the Program Counter.

3.2.1 - Flag Register

Flag Registers will be established to allow the CPU to store the ALU flags at the end of each clock cycle.



Subsystem 1 - Flag Register

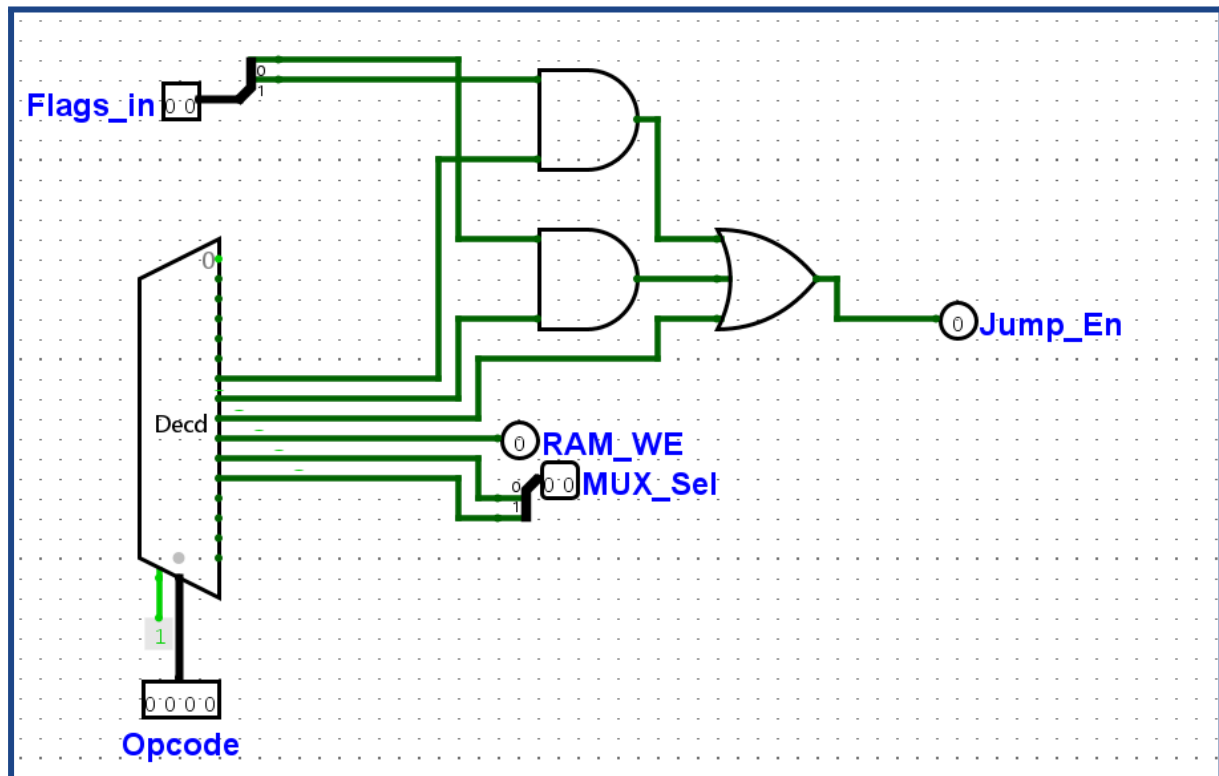
3.2.2 - Control Unit

Implementation of the Jump If Zero (JZ) instruction. The JZ instruction enables conditional branching, allowing the CPU to jump to a specific address only when the previous ALU operation results in zero ($Z_Flag == 1$).

- **Flag Register:** A 2-bit register synchronized with the clock (CLK) captures and holds the Z and C flags coming directly from the ALU.
- **Control Unit Routing:** The stored flags are fed into the Control Unit via a dedicated Flags_in input pin.

- **Condition Check (AND Gate):** An AND gate combines the Z_Flag with the decoded JZ instruction signal, ensuring the jump condition is met only when the ALU output is actually zero.
- **Jump Unification (OR Gate):** An OR gate merges this conditional JZ signal with the standard unconditional JMP signal, driving the final Jump_En line for the Program Counter.

In a very similar way, we implement the Jump if Carry (JC) instruction. Both are demonstrated below:



Subsystem 2 - JZ/JC through the Control Unit

- **ISA Update - JZ/JC logic:**
 - ◆ **New Opcode Added:** Opcodes 7 and 6 are now officially reserved in the ISA for the JZ (Jump if Zero) and JC instructions, respectively.
 - ◆ **Conditional Execution:** The CPU gains conditional branching capabilities, allowing it to make dynamic runtime decisions on whether to jump based on the ALU's Z_Flag and C_Flag.
 - ◆ **Format 2 Alignment:** It integrates into the second instruction format, combining Opcode 7 with the target jump address, where execution is gated by the hardware flag logic (Similarly for JC).

3.3 - Full RAM Integration

The integration of Random-Access Memory (RAM) completes the processor core architecture, enabling general-purpose data storage, variable management, and dynamic execution of complex software algorithms. This subsystem establishes a classic Harvard, Load-Store RISC model that decouples data memory access from arithmetic processing.

- **Address Bus Connections (Pin A):** The RAM address input is connected to the Out_B port of the Register File, implementing Register Indirect Addressing to dynamically select target memory locations for read and write operations.
- **Data Input Connections (Pin D_in):** The memory data input is driven directly by the Out_A port of the Register File, isolating memory write transfers from the ALU execution pipeline.

Control Interfacing & Bus Protection: The Write Enable (WE) line is controlled by the Control Unit, asserting write access strictly during STORE instruction execution (Opcode 9). During LOAD instructions (Opcode A), the central multiplexer selects the RAM data output bus to update the target register while preventing bus contention.

4. Testing and Finalizing

Following the completion of the CPU core design, an initial verification phase is conducted to evaluate the system's functional capabilities and validate its support for general-purpose algorithmic execution. The Fibonacci sequence algorithm was selected as the primary benchmark test, as it comprehensively exercises key architectural components—including arithmetic computations, data memory utilization, Load Immediate (LDI) operations, and conditional control logic (branching). Additionally, this process serves as the initial physical verification of the Instruction Set Architecture (ISA) and ROM instruction encoding, confirming the effectiveness of the control logic in preventing data bus collisions.

The following logic will be implemented:

- The register file will be used to store past values, determine RAM's initial address, and store a constant 1 to push the address value, so that the RAM can store the entire sequence. Specifically,
 - ◆ R0 will store even terms (F_0, F_2, F_4, ...)
 - ◆ R1 will store odd terms (F_1, F_3, F_5, ...)
 - ◆ R2 will act as a RAM pointer beginning on address 128
 - ◆ R3 = 1 to push R2 from 128 onwards
- The table below details the ROM encoding and the rationale for each instruction.

Address (Hex)	Hex Code	Assembly Instruction	Description / Notes
00	B801	LDI 1 (R0)	Loads value 1 into register R0.
01	0BC0	OR R0, R0 -> R3	Copies value 1 to R3 (constant increment value).
02	0B40	OR R0, R0 -> R1	Copies value 1 to R1 ($F_1 = 1$).
03	B880	LDI 128 (R2)	Loads RAM start pointer address (128) into R2.
04	B800	LDI 0 (R0)	Sets value 0 into R0 ($F_0 = 0$).
05	9008	STORE R0, (R2)	$RAM[128] = 0$.
06	08AC	ADD R2, R3 -> R2	Increments RAM pointer ($R2 = 129$).
07	9018	STORE R1, (R2)	$RAM[129] = 1$.
---	---	---	--- START OF LOOP ---
08	0804	ADD R0, R1 -> R0	Calculates next term: $R0 = R0 + R1$.
09	6011	JC 11	If Carry occurs (overflow > 255), jump to HALT state.
0A	08AC	ADD R2, R3 -> R2	Increments RAM pointer ($R2++$).
0B	9008	STORE R0, (R2)	Stores newly calculated term into RAM.
0C	0850	ADD R1, R0 -> R1	Calculates next term: $R1 = R1 + R0$.
0D	6011	JC 11	If Carry occurs, jump to HALT state.
0E	08AC	ADD R2, R3 -> R2	Increments RAM pointer ($R2++$).
0F	9018	STORE R1, (R2)	Stores newly calculated term into RAM.
10	8008	JMP 08	Jump back to loop start (Line 08).
---	---	---	--- HALT STATE ---
11	8011	JMP 11	Infinite Loop: Halts execution.

Table 7 - Benchmark Test ROM Program Encoding (Fibonacci)

5. Peripherals and Software

Having established a fully functional, general-purpose CPU core capable of algorithmic execution, this section details the integration of dedicated peripheral hardware and software design required for real-time application execution on FPGA. The integration encompasses three core modules:

- **Memory-Mapped I/O Interfacing:** Mapping hardware controls (FPGA switches/buttons) and video RAM (VGA display driver) into the unified memory address space.
- **State & Variable Allocation:** Structuring data structures in RAM and register files to track spatial coordinates, velocities, and player scores.
- **Game Control Loop Logic:** Developing machine-level assembly routines to process player input, evaluate collision physics, update game states, and trigger screen refreshes.

5.1 - Memory-Mapped I/O

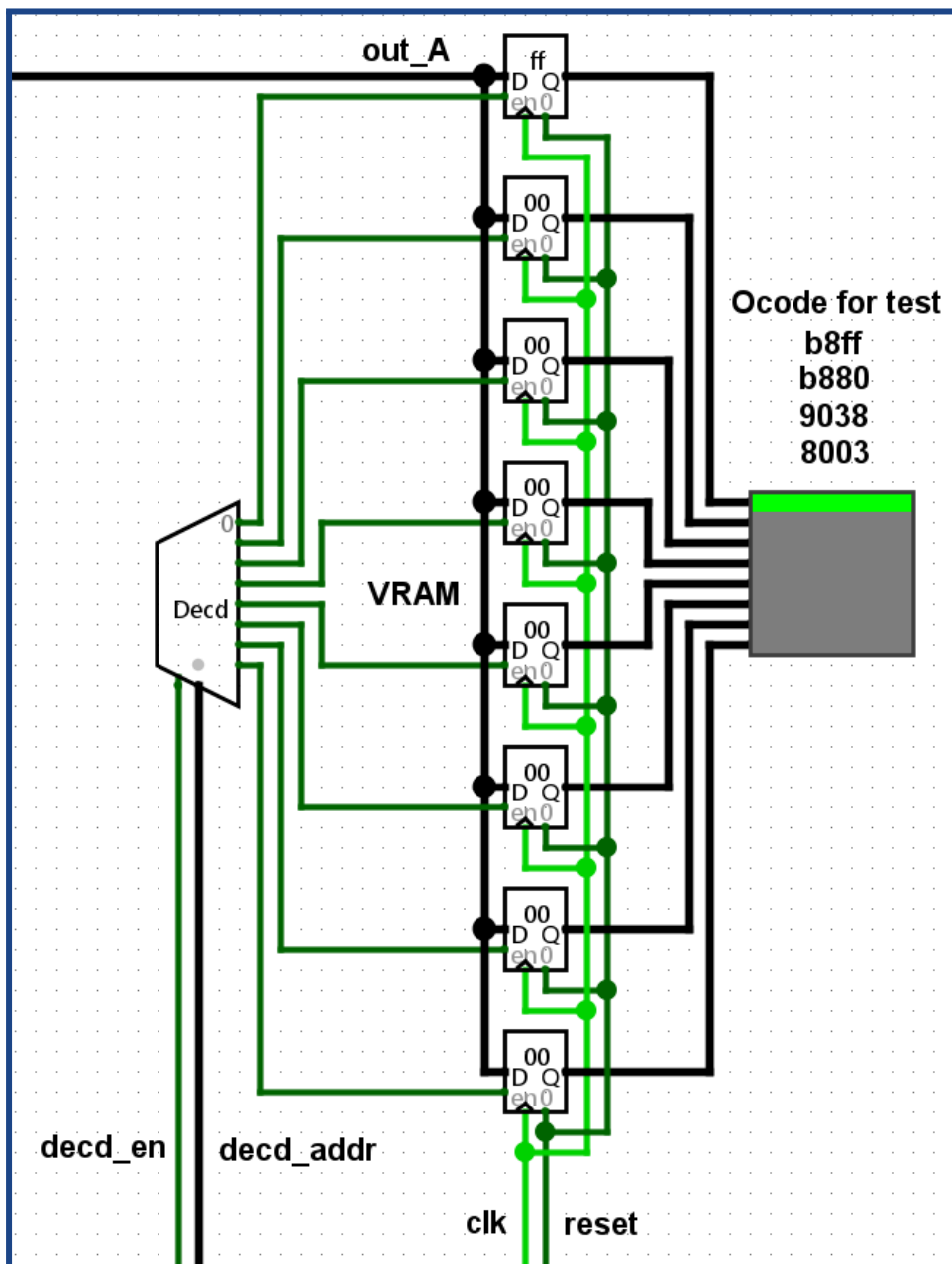
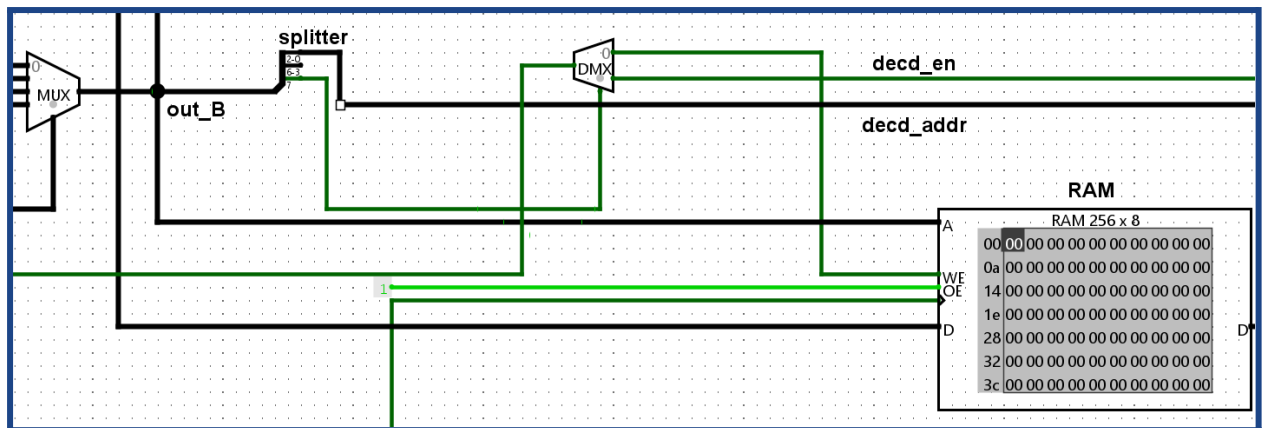
In 8-bit CPU design, memory-mapped I/O (MMIO) is an architectural approach where peripheral devices—like keyboards, display controllers, and sound chips—are assigned specific addresses within the CPU's standard memory space.

Rather than giving the CPU separate "doorways" for memory and hardware, everything shares the exact same address bus and data bus.

5.1.3 - LED Matrix

The LED Matrix management subsystem offers the 8-bit CPU to project graphics on an 8x8 discrete screen. Its function relies on a Memory-Mapped I/O technique, that splits the memory storage in half, to share between the CPU and the matrix.

- **Address Decoding:** The CPU outputs an 8-bit address. The system uses a Splitter to isolate **Bit 7** (the Most Significant Bit). If Bit 7 is 0 (addresses 0x00-0x7F), the CPU communicates with standard RAM. If Bit 7 is 1 (addresses 0x80-0xFF), the CPU communicates with the display (VRAM).
- **Write Routing (Demultiplexer):** The central CPU write enable signal (WE) passes through a Demux controlled by Bit 7. This creates a new, dedicated signal, **VRAM_WE**, which is activated *only* when the CPU intends to write to the screen display.
- **Display Memory (8 Registers & Decoder):** The "VRAM" consists of 8 independent 8-bit registers (one for each row of the LED Matrix). Register selection is handled by a **Decoder** reading **Bits 0-2** of the address. The enable pin (en) of this Decoder connects directly to VRAM_WE. Consequently, the registers accept new data only when an explicit write command to VRAM is issued, protecting the display from accidental writes ("glitches") during read cycles or execution of other code.

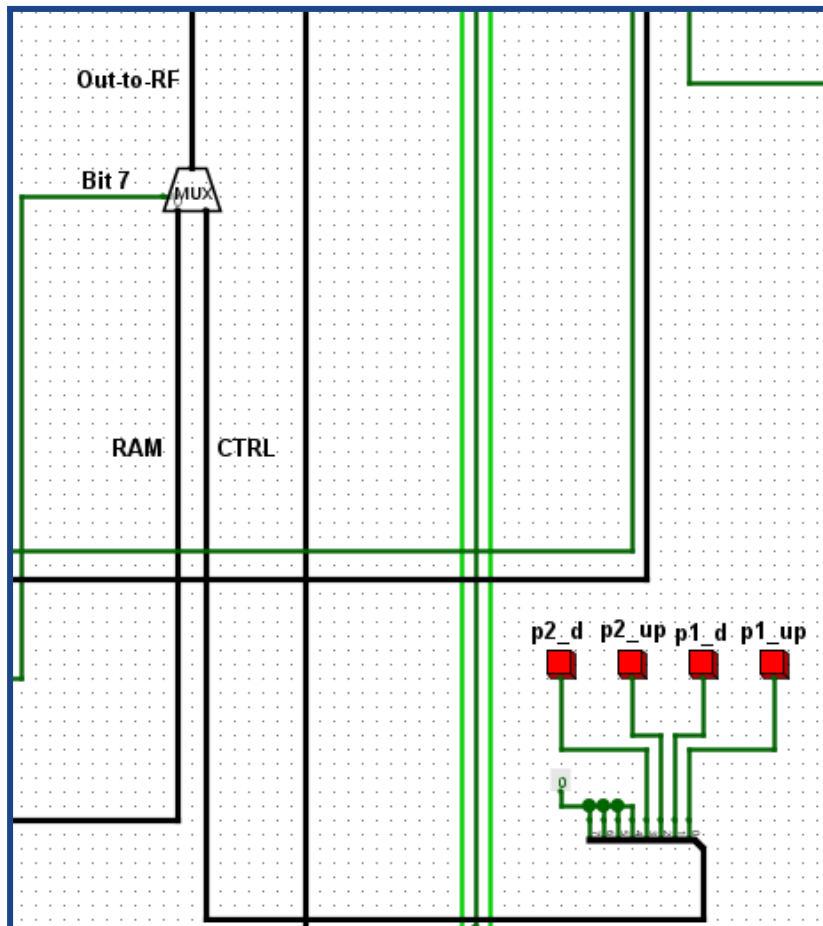


Subcircuit 5 - LED Matrix Management

Note: decd_addr consists of the 3 least significant digits in out_B

5.1.4 - Controller

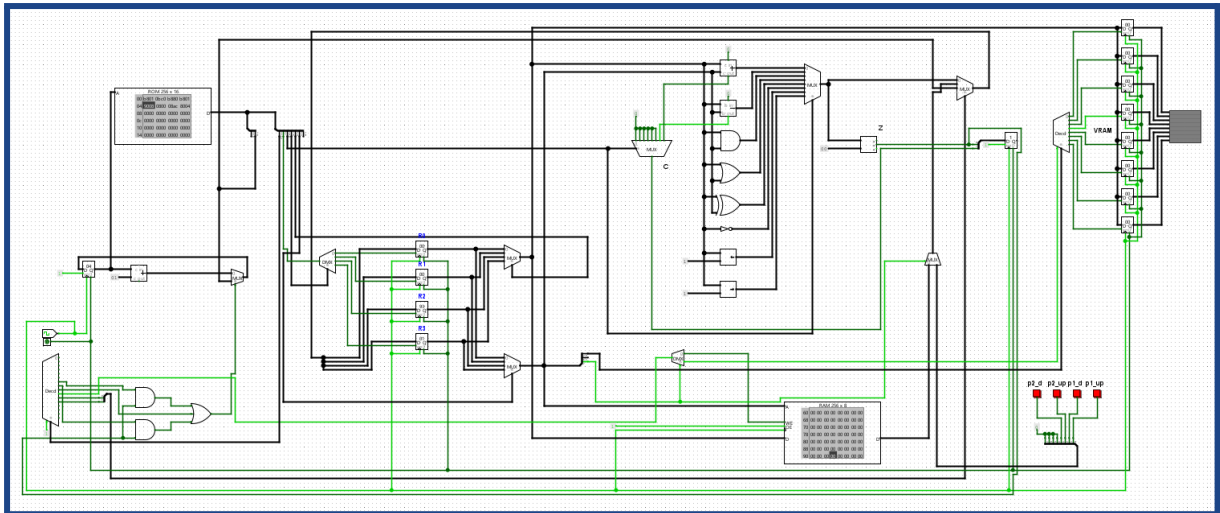
The controls system will be implemented through a four-button controller, retaining the Memory-Mapped input technique. The input from the buttons will conditionally override the RAM through a multiplexer, and send the necessary encodings to the register file, allowing for the correct output to be displayed onto the LED Matrix. The exact implementation is showcased in the schematics below.



Subsystem 3 - Controller

At this stage, the hardware design is complete. All core subsystems have been verified through functional testing, and the Instruction Set Architecture (ISA) is fully specified without data bus collisions. Additionally, the necessary peripheral interfaces, including the LED matrix display and button controllers, have been integrated to support generic interactive applications. The remainder of this report focuses on the machine-level software development required to execute software routines.

It is worth noting that the CPU's general-purpose capabilities allow it to execute various software algorithms and control loops beyond basic hardware verification tests.



Circuit 3 - Complete CPU core Design

5.2 - Software Development

With the physical CPU architecture, Memory-Mapped I/O interfaces, and ISA fully established, executing complex software routines requires dedicated programs aligned with the underlying hardware constraints. Writing raw hexadecimal machine code directly into ROM is error-prone and inefficient for larger software logic. To streamline this, a software abstraction layer is developed using Python to bridge high-level programming with the CPU's custom Assembly language.

The Python assembler class (Asm) encapsulates the Instruction Set Architecture by defining helper methods for every hardware instruction (e.g., alu, load, store, jmp, jz, jc). It dynamically tracks jump labels, formats instruction bitfields according to the specified 16-bit ROM layer distribution, and handles immediate value routing (LDI stage/copy logic). This allows game developers to structure software using readable Assembly-style syntax in Python, which is then automatically assembled into valid binary/hexadecimal machine code for hardware deployment. (The following code is a guided Claude product)

```
R0, R1, R2, R3 = 0, 1, 2, 3
REGNAME = {R0: "R0", R1: "R1", R2: "R2", R3: "R3"}

ADD, SUB, AND, OR, XOR, NOT, SHL, SHR = range(8)
ALUNAME = {ADD: "ADD", SUB: "SUB", AND: "AND", OR: "OR", XOR: "XOR",
            NOT: "NOT", SHL: "SHL", SHR: "SHR"}

class Asm:
```

```

def __init__(self):
    self.prog = []          # list of dicts describing each real
instruction
    self.labels = {}        # name -> address (index)
    self.pending_jumps = [] # (index, label, kind)

    # ---- low level emit ----
    def _emit(self, **kw):
        self.prog.append(kw)
        return len(self.prog) - 1

    def here(self):
        return len(self.prog)

    def label(self, name):
        assert name not in self.labels, f"duplicate label {name}"
        self.labels[name] = self.here()

    def comment(self, text):
        # attach a standalone comment (no-op instruction marker not
needed;
        # we just stash it to print later against the *next*
instruction)
        self._next_comment = text

    def _c(self):
        c = getattr(self, "_next_comment", None)
        self._next_comment = None
        return c

    # ---- real instructions ----
    def alu(self, op, ra, rb, wd, we=True, note=None):
        self._emit(kind="ALU", op=op, ra=ra, rb=rb, wd=wd, we=we,
                    note=note or self._c())

    def cmp(self, ra, rb, note=None):
        """SUB ra,rb, discard result, only used for the Z flag."""
        self._emit(kind="ALU", op=SUB, ra=ra, rb=rb, wd=0, we=False,
                    note=note or self._c() or f"CMP
{REGNAME[ra]}, {REGNAME[rb]}")

    def store(self, ra_data, rb_addr, note=None):

```

```

        self._emit(kind="STORE", ra=ra_data, rb=rb_addr, note=note or
self._c())

    def load(self, rd, rb_addr, note=None):
        self._emit(kind="LOAD", rd=rd, rb=rb_addr, note=note or
self._c())

    def jmp(self, label, note=None):
        self._emit(kind="JMP", label=label, note=note or self._c())

    def jz(self, label, note=None):
        self._emit(kind="JZ", label=label, note=note or self._c())

    def jc(self, label, note=None):
        self._emit(kind="JC", label=label, note=note or self._c())

    def ldi_raw(self, value, note=None):
        """Emit a single real LDI. Destination is forced by value's
magnitude."""
        value &= 0xFF
        self._emit(kind="LDI", value=value, note=note or self._c())
        return value >> 6 # register that received it

    def imm(self, value, dst, note=None):
        """Load an 8-bit immediate into an arbitrary register dst.
        Emits 1 instruction if dst already matches the value's natural
        register, otherwise 2 (LDI to natural reg + OR-copy to dst)."""
        value &= 0xFF
        natural = value >> 6
        tag = note or self._c()
        if natural == dst:
            self.ldi_raw(value, note=tag)
        else:
            self.ldi_raw(value, note=(tag + " (stage)") if tag else
None)
            self.alu(OR, natural, natural, dst,
                    note=(tag + " (copy)") if tag else f"copy ->
{REGNAME[dst]}")

    def mov(self, src, dst, note=None):
        self.alu(OR, src, src, dst, note=note or self._c() or f"MOV
{REGNAME[src]}->{REGNAME[dst]}")

```

```

# ---- assembly ----
def assemble(self):
    words = []
    for i, ins in enumerate(self.prog):
        k = ins["kind"]
        if k == "ALU":
            w = (0 << 12) | ((1 if ins["we"] else 0) << 11) |
(ins["op"] << 8) \
                | (ins["wd"] << 6) | (ins["ra"] << 4) | (ins["rb"]
<< 2)

            elif k == "STORE":
                w = (0x9 << 12) | (ins["ra"] << 4) | (ins["rb"] << 2)
            elif k == "LOAD":
                w = (0xA << 12) | (1 << 11) | (ins["rd"] << 6) |
(ins["rb"] << 2)
            elif k == "LDI":
                w = (0xB << 12) | (1 << 11) | ins["value"]
            elif k in ("JMP", "JZ", "JC"):
                addr = self.labels[ins["label"]]
                assert 0 <= addr <= 0xFF, f"jump target out of range:
{ins['label']}={addr}"
                opc = {"JMP": 0x8, "JZ": 0x7, "JC": 0x6}[k]
                w = (opc << 12) | addr
            else:
                raise ValueError(k)
            words.append(w & 0xFFFF)
    return words

def listing(self, words):
    lines = []
    addr_to_label = {v: k for k, v in self.labels.items()}
    for i, (ins, w) in enumerate(zip(self.prog, words)):
        lab = addr_to_label.get(i, "")
        mnem = self._mnemonic(ins)
        note = ins.get("note") or ""
        lines.append(f"{i:3d} {0x{i:02X}} {w:04X} {lab:14s}
{mnem:28s} ; {note}")
    return "\n".join(lines)

def _mnemonic(self, ins):
    k = ins["kind"]
    if k == "ALU":
        we = "" if ins["we"] else " (flags only)"

```

```

        return f"{ALUNAME[ins['op']]}"
{REGNAME[ins['ra']], {REGNAME[ins['rb']] } -> {REGNAME[ins['wd']] } {we} "
    if k == "STORE":
        return f"STORE {REGNAME[ins['ra']],
({REGNAME[ins['rb']]}) "
    if k == "LOAD":
        return f"LOAD {REGNAME[ins['rd']], ({REGNAME[ins['rb']]}) "
    if k == "LDI":
        v = ins["value"]
        return f"LDI 0x{v:02X} (-> {REGNAME[v>>6]}) "
    if k in ("JMP", "JZ", "JC"):
        return f"{k} {ins['label']}"
    return "?"

```

→ This example code lights up a pixel if its corresponding button is pressed.

```

from cpu import Asm, R0, R2
a = Asm()
a.label("LOOP")
a.imm(0x80, R2, note="R2 = 0x80 (Controller Input / VRAM Row 0)")
a.load(R0, R2, note="Store the buttons at R0")
a.store(R0, R2, note="Light up the corresponding pixel through VRAM")
a.jump("LOOP", note="loop")
# Hex Code Production
words = a.assemble()
print("v2.0 raw")
for word in words:
    print(f"{word:04X}")

```

6. Summary and Future Goals

To conclude the presented report, this project successfully demonstrates the design and verification of a custom 8-bit RISC CPU architecture tailored for specialized execution environments. The processor's capabilities include fundamental arithmetic computations (such as addition, subtraction, logic operations, bitwise shifting, and integer multiplication), and executing algorithms, supported by full RAM/ROM integration and status flag logic for conditional branching. Through its unified memory-mapped I/O interface, the core seamlessly interacts with external hardware peripherals, reading real-time inputs via physical button controllers and driving graphical output directly to an 8x8 LED matrix display.

Looking ahead, the primary objective is to construct and assemble a fully featured, interactive two-player Pong game using our custom Python-assembler, validating complex game loops, collision physics, and rendering logic directly in machine code. To finalize the project, the ultimate goal is to manufacture the verified circuit design onto a physical FPGA development board, completing the transition from simulated logic to a standalone, hardware-implemented arcade gaming platform.